MAQAM / SOFTWARE VERSION 0.3.3

Exact-input governance for registered AI-agent actions

An implementation-backed account of policy-before-dispatch, exact one-use approval, execution receipts, evidence links, adapters, and their limits.

Author: Ajnas N B
Paper version: 1.0
Date: 8 August 2026
Implementation: v0.3.3 / f43c2493084f8a6c8c755a50a3d9feb38d72ebcc

DOI: Not assigned
Status: Project-authored technical report. No independent peer review, formal verification, penetration test, or independent security certification is claimed.

Contents

Architecture, authority, adapters, verification, measurements, and explicit limits.

Abstract

Software agents increasingly cross a boundary between proposing work and causing side effects. A model may select a function, command-line worker, browser action, network reader, or internal service, while the host application must decide whether the proposed operation is allowed and whether a human approval still covers the exact input that will execute. Maqam is an open-source TypeScript governance layer for operations deliberately routed through a registered gateway. Its implementation combines pre-dispatch policy evaluation, canonical input snapshotting, approval bound to the run, tool, and input hash, one-use consumption by default, bounded trace records, and explicit evidence links.

This paper documents the architecture and authority model implemented by Maqam 0.3.3 at immutable Git tag `v0.3.3`. It separates implementation claims from deployment assumptions and historical measurements. The project-defined Maqam Governance Evaluation Suite (MGES) provides deterministic conformance fixtures and a narrow local-call microbenchmark. The latest measured-source result discussed here belongs to the 0.3.1 implementation line: 14 of 14 project-defined conformance fixtures passed, and the local governed-call fixture recorded a median of 129.849 microseconds per call on the disclosed environment. That result does not measure model inference, network or filesystem I/O, durable storage, browser behavior, human review, concurrency, or production capacity, and it is not relabeled as a 0.3.3 result.

The central conclusion is deliberately bounded: Maqam can make the authority path for registered operations explicit and inspectable, but it cannot govern calls that bypass that path, authenticate a reviewer, secure an injected driver, replace operating-system isolation, or prove that a claim linked to evidence is true.

1. Problem statement

An agent request usually contains at least four distinct objects: an objective, a selected operation, an input payload, and an authority decision. Treating those objects as one natural-language prompt creates ambiguity. The words a reviewer sees can differ from the bytes a tool receives, the tool can be called again after approval, or the host can bypass the reviewed path entirely.

Maqam addresses a narrower engineering question: how can a TypeScript host place a verifiable governance boundary around operations it explicitly registers? The design objectives are:

- evaluate policy before the registered handler runs;
- bind approval to the exact run, tool, and canonical input;
- consume approval before dispatch and reject replay by default;
- keep registered effects and network origins authoritative over model-controlled input;
- produce bounded execution records and explicit claim-to-evidence links;
- reject hostile authority objects instead of invoking accessors or trusting inherited state;
- expose adapters without claiming to secure the external systems they invoke.

The non-objective is equally important. Maqam is not a syscall interceptor, model provider, operating-system sandbox, identity system, credential vault, durable database, browser engine, distributed workflow service, or compliance certification.

2. Artifact identity and method

This report uses the immutable source artifact below:

```
repository: https://github.com/AjnasNB/maqam
tag:        v0.3.3
commit:     f43c2493084f8a6c8c755a50a3d9feb38d72ebcc
package:    maqam@0.3.3
runtime:    Node.js 22, 24, or 26
license:    MIT
```

The tag target was verified against the remote Git reference. Architectural statements were checked against the tagged source, especially `src/framework/tool-gateway.js`, `policy.js`, `approval-queue.js`, `boundary.js`, `audit.js`, `evidence-ledger.js`, `runtime.js`, the adapter implementations, and the tagged feature inventory. Release identity remains a multi-part claim: the Git tag, npm `gitHead`, tarball integrity, provenance record, and GitHub release should be checked independently before adoption.

The report distinguishes four evidence classes:

1. **Tagged implementation evidence** describes code present at `v0.3.3`.
2. **Project test evidence** describes repository tests run against a disclosed checkout.
3. **Historical measured-source evidence** remains attached to the exact source fingerprint that produced it.
4. **Deployment assertions** belong to the host and are not inferred from the library.

This is a design and implementation report, not an independent evaluation. No external party audited the implementation for this paper.

3. Architecture

Maqam separates the host's intent, governance decision, execution, and evidence surfaces. The core path can be summarized as follows:

```
Host objective
     |
     v
PolicyEngine -- allow / deny / needs_approval
     |
     v
ToolGateway -- registration, input snapshot, limits, trace
     |
     +--> ApprovalQueue -- exact run + tool + input hash, consume once
     |
     v
Registered handler or adapter
     |
     +--> execution receipt
     +--> EvidenceLedger -- explicit evidence and claim links
```

3.1 Policy engine

`PolicyEngine` evaluates goal and tool-call constraints. Its configuration can restrict tools, effects, origins, total calls, goal budgets, and tenant scope. Decisions use the explicit states `allow`, `deny`, and `needs_approval`. Policy-owned limits cannot be raised by caller-supplied context. When configured and requested scopes overlap, Maqam narrows them by intersection; an empty intersection is treated as a conflict rather than silent expansion.

The policy engine is an application-side decision component. A correct decision does not stop a caller from invoking an unregistered function directly. Enforcement depends on the host routing the operation through `ToolGateway`.

3.2 Tool gateway

`ToolGateway` requires explicit registration before dispatch. A registration records the handler and immutable metadata such as effects, origins, and risk. At call time, the gateway snapshots context and input, evaluates policy, computes the effective limits, verifies approval when required, records trace state, and invokes the registered handler.

The gateway does not accept model-controlled metadata as the source of truth for registered effects. It also exposes guarded-tool construction for operations that must only execute during an active gateway dispatch. These checks reduce accidental bypass inside the integration, but they do not constitute an operating-system boundary.

3.3 Boundary snapshots

Authority-bearing input is treated as data, not as an arbitrary JavaScript object graph. Tagged boundary helpers reject accessors, inherited enumerable authority, prototype-sensitive keys, cycles where disallowed, unsupported values, excessive depth, and excessive collection size. Accepted records are detached from caller-owned references and frozen.

This protects the governance calculation from a class of time-of-check/time-of-use and hostile-object problems. It does not make downstream external systems deterministic, and it does not sanitize every domain-specific string a handler may interpret.

3.4 Approval queue

`ApprovalQueue` stores deterministic approval records and supports request, approve, reject, single consumption, atomic multi-consumption, serialization, and restoration. An approval subject can bind the request to an exact run, tool, and canonical input hash. Consumption records who consumed the approval and the relevant run and tool identity.

One-use behavior is the default. Reusable approval is an explicit mode and should be treated as a broader authority grant. Reviewer identity is host-supplied data; Maqam does not authenticate the reviewer or provide non-repudiation.

3.5 Evidence ledger

`EvidenceLedger` stores normalized evidence and claims with stable identifiers and content hashes. Claims reference evidence identifiers, and scoped facades preserve run and task attribution. Batch operations allow related evidence and claims to be added atomically within the in-memory ledger.

An evidence link establishes provenance inside the recorded workflow. It does not prove factual correctness, relevance, freshness, or completeness. The host must supply durable storage if records need to survive a process or satisfy retention requirements.

3.6 Workflow runtime

`AgentRuntime` executes sequential workflow tasks with task identifiers, retry policy, timeouts, cancellation, goal and tenant ceilings, approval pause and resume, traces, and scoped evidence access. Deterministic governance errors are excluded from automatic retry. Active run-id collision and duplicate task-id checks reduce ambiguous attribution.

The runtime is not a distributed orchestrator. It does not provide a consensus protocol, leased workers, durable queues, exactly-once distributed delivery, or recovery after host failure.

4. Exact-input approval protocol

The approval protocol is designed around a simple invariant: the authority a reviewer grants should describe the same operation the handler receives.

4.1 Canonicalization and hashing

Maqam snapshots accepted JSON-like input and computes a stable hash over its canonical representation. Approval scope includes the run identity, tool name, and input hash. Canonicalization avoids treating object key insertion order as a different semantic payload while rejecting object forms that could execute code during inspection.

4.2 Decision before dispatch

Policy evaluation occurs before the registered handler. A denial or policy evaluation failure stops dispatch. If approval is required, the gateway verifies that the supplied approval belongs to the same run and tool and covers the exact input hash.

4.3 Consumption before effect

The gateway consumes the matching approval before it invokes an approval-gated handler. A changed input fails scope validation, and a consumed approval cannot be replayed unless the approval was explicitly marked reusable. Grouped operations can request atomic multi-approval consumption so a partial set is not consumed when the full set cannot be validated.

4.4 Residual race boundary

Approval consumption is not transactionally coupled to an arbitrary external side effect. A process can fail after consumption but before the external system confirms the operation. Conversely, an external system can accept a request and fail before a final receipt is persisted. Integrations that require atomic external effects need domain-specific idempotency keys, transactional outboxes, or equivalent host infrastructure.

5. Adapter surfaces and trust boundaries

Maqam exposes multiple ways to bring existing workers into the same registered call path. Each adapter narrows one integration surface; none erases the trust boundary of the external component.

5.1 Function and object agents

Function agents and objects exposing `run`, `invoke`, or `call` can be wrapped as governed tools. The adapter passes bounded run, task, goal, evidence, and cancellation context. The gateway remains authoritative for governance records, so worker output cannot overwrite the dispatch decision.

5.2 Generic tool adapters

Data-first adapter definitions support function, SDK, HTTP, MCP-style, and custom transports. Maqam validates definitions and can produce conformance reports. The host still supplies discovery, authentication, transport clients, and service-specific authorization. Maqam is not a universal MCP client, MCP server, or plugin marketplace.

5.3 Command-line workers

Command-line adapters use a fixed executable and argument configuration without shell evaluation. They parse JSON Lines events and can bound duration, output bytes, events, estimated tokens, environment forwarding, and working-directory scope. Symlink-aware containment checks are part of the local path boundary.

Codex CLI and Claude Code adapters normalize provider events and apply documented default safety arguments. Those adapters govern the configured process invocation; they do not intercept actions performed outside it, and provider-reported token usage may be available only after execution.

5.4 Governed browser contract

The browser contract separates `observe`, `preview`, `apply`, and `submit`. Model-controlled plans use opaque page, element, value, revision, and plan identities rather than raw selectors, raw scripts, or raw secret values. Expected origin, new-page behavior, stale revision handling, and driver attestations constrain the contract.

The injected browser driver is trusted. Maqam does not bundle Chromium, Playwright, profiles, a browser sandbox, or rollback. A malicious or defective driver can misreport its actions.

5.5 Research routing and crawler

`ResearchSourceRegistry` orders validated adapters and routes them through a host-supplied gateway caller. Fallback is limited to classified unavailable or allowed failure states; authentication, policy, and security failures stop fallback. Built-in adapters cover host-supplied crawling, offline RSS and Atom parsing, optional hosted-anonymous Exa search, and optional `yt-dlp` YouTube metadata and available captions under explicit limits.

The built-in crawler supports bounded public HTTP(S) traversal, origin controls, robots handling, sitemap and feed discovery, redirects, DNS classification, request and byte limits, and normalized records. It does not execute page JavaScript or bypass authentication, authorization, CAPTCHA, paywalls, or robots directives. The separate Cockroach Crawler project has a broader crawling surface and is not silently included in Maqam.

6. Threat relevance and deployment model

The OWASP Top 10 for Agentic Applications 2026 identifies tool misuse, identity and privilege abuse, and human-agent trust exploitation among broader agentic risks. Maqam's registered boundary is relevant to parts of those risks: pre-dispatch policy constrains registered tools, exact approval limits authority reuse, and trace records support review. This relevance is not OWASP compliance, endorsement, or full coverage.

Important deployment controls remain outside the library:

- authenticate users and reviewers, and authorize their roles;
- isolate untrusted workers with an operating-system sandbox, container, or virtual machine;
- store credentials in a dedicated secret system and pass only narrowly scoped references;
- restrict network egress and validate destination policy outside model control;
- provide durable, access-controlled storage for approvals, traces, and evidence;
- implement service-specific idempotency, rollback, or compensation;
- monitor direct calls and integration paths that can bypass the gateway;
- assess prompt injection, data poisoning, social engineering, and reviewer fatigue.

Maqam should be placed close to the side-effect boundary, after the host has authenticated the caller and before the registered external action is dispatched.

7. Verification evidence

The repository contains unit, integration, adversarial, CLI, server, crawler, source, browser, approval, evidence, runtime, release, and documentation tests. Its verification command combines the Node test suite, a clean packed-consumer TypeScript compile, website validation, production dependency audit, and npm tarball dry run. Continuous integration targets maintained Node.js 22, 24, and 26 releases and includes CodeQL analysis.

Passing project tests supports a regression claim for the tested source and environment. It does not establish the absence of defects, malicious dependencies, unsafe host configuration, or vulnerabilities outside test coverage.

7.1 Supply-chain identity

The public 0.3.3 package record states that the release used npm Trusted Publishing with provenance. npm documents that trusted publishing exchanges CI identity through OpenID Connect and can automatically generate provenance for supported public GitHub or GitLab workflows. npm also states that provenance links a package to source and build instructions but does not guarantee the package is free of malicious code.

Consumers should verify the current package directly:

```
npm view maqam@0.3.3 version gitHead dist.integrity
npm view maqam dist-tags.latest
npm audit signatures
```

The expected `gitHead` for the paper's software artifact is `f43c2493084f8a6c8c755a50a3d9feb38d72ebcc`. Registry and source records can change in availability, so verification should occur at adoption time.

8. MGES evaluation

MGES v1.1.0 is a project-defined evaluation suite with two separate profiles. It is not a globally standardized benchmark, an industry certification, a competitor ranking, a penetration test, or a security score.

8.1 Governance-boundary conformance

The conformance profile exercises named deterministic invariants. The measured-source 0.3.1 artifact records 14 of 14 fixtures passing on clean commit `a96413c4da5f27dc31b9772996e70faab0b38382`. The cases include denial before dispatch, fail-closed policy behavior, rejection of accessor-bearing input, exact run/tool/input approval scope, one-use consumption, policy-owned call limits, detached frozen input, evidence scoping, atomic multi-approval consumption, bounded trace redaction, source denial without fallback, and normalized ordered fallback.

The profile reports pass or fail rather than a weighted score. It does not test every browser behavior, prompt injection, identity infrastructure, operating-system isolation, deployment configuration, or semantic correctness.

8.2 Local-call performance

The historical 0.3.1 measured-source local-call profile alternates direct and governed variants across 30 fresh child processes per variant. Each governed observation executes 5,000 sequential calls after 500 warmup calls. Timing uses Node's monotonic `process.hrtime.bigint()` API. A deterministic 10,000-resample percentile bootstrap estimates a 95% interval for the sample median.

The disclosed GitHub-hosted environment was Node 24.18.0 on Ubuntu 24.04 x64 with an AMD EPYC 7763. The governed path recorded:

- median: 129.849 microseconds per call;
- 95% bootstrap interval for the sample median: 129.539 to 130.648 microseconds;
- governed coefficient of variation: 1.111 percent;
- sequential reciprocal rate at the median: 7,701.249 calls per second;
- project publication checks: PASS;
- measured source: clean commit `a96413c4da5f27dc31b9772996e70faab0b38382`.

The timed fixture is an in-process asynchronous handler returning `input.value + 1`. The governed variant adds input snapshotting and hashing, policy evaluation, limits, a redacted trace, and dispatch. It excludes model inference, network I/O, filesystem I/O, durable storage, human review, process startup, browser actions, and concurrent load. The reciprocal rate is not a throughput or production-capacity result.

8.3 Interpretation limits

The MGES result is useful for regression tracking under matched conditions. It does not justify a cross-product speed claim because other products were not configured to execute the same policy, payload, approval, trace, persistence, and evidence obligations. It does not justify a 0.3.3 point estimate because 0.3.3 did not publish a new measured-source MGES run. The paper therefore retains the result under its historical source identity.

NIST AI 800-2, released as an Initial Public Draft in January 2026, recommends transparent objectives, assumptions, uncertainty, evaluation details, and item-level results for automated benchmark evaluations. MGES borrows compatible reporting practices, but the draft primarily addresses language-model and agent-system evaluations and does not certify MGES or Maqam.

9. Limitations

The most important limitations are structural rather than cosmetic:

- **Bypass remains possible.** A host can call a function, process, network client, or browser driver outside the gateway.
- **Reviewer identity is external.** Approval records contain host-supplied identity strings, not authenticated signatures.
- **External effects are not atomic with receipts.** A crash can occur between consumption, dispatch, external commit, and durable recording.
- **Drivers and adapters remain trusted.** Maqam validates contracts but cannot prove that an injected component reports truthfully.
- **Evidence is provenance, not truth.** A claim can cite evidence that is incomplete, stale, or misinterpreted.
- **In-memory state is not durable infrastructure.** Production retention and concurrency require host-owned storage and coordination.
- **Browser and crawler coverage is bounded.** The library is neither a browser engine nor a distributed crawler fleet.
- **MGES is narrow.** It cannot predict end-to-end latency, production capacity, security posture, or reviewer effectiveness.
- **No independent audit is claimed.** This paper is authored within the project and should be evaluated accordingly.

10. Reproduction protocol

The following protocol checks the paper's implementation anchor without mutating a production environment:

1. Clone `https://github.com/AjnasNB/maqam`.
2. Fetch the immutable `v0.3.3` tag.
3. Verify that the tag resolves to `f43c2493084f8a6c8c755a50a3d9feb38d72ebcc`.
4. Check out the tag in a detached worktree.
5. Install exactly locked dependencies with `npm ci` on a supported Node release.
6. Run `npm run verify`.
7. Run `npm run benchmark:mges:conformance` if a fresh local conformance observation is required.
8. Treat any new performance run as a new environment-specific artifact, not as reproduction of the published point estimate unless source fingerprints and protocol match.
9. Verify the npm artifact and provenance independently with registry tooling.

Example commands:

```
git fetch origin tag v0.3.3
git rev-list -n 1 v0.3.3
git worktree add ../maqam-v0.3.3 v0.3.3
cd ../maqam-v0.3.3
npm ci
npm run verify
npm run benchmark:mges:conformance
npm view maqam@0.3.3 version gitHead dist.integrity
```

Re-running a benchmark on a different host is valuable independent evidence. It should disclose runtime, operating system, processor, source hash, configuration, raw observations, uncertainty, and failed attempts.

11. Conclusion

Maqam 0.3.3 implements a focused governance path for registered AI-agent operations in TypeScript. Its strongest property is not universal control; it is explicitness. Policy is evaluated before a registered handler, approval can be bound to the exact input and consumed once, authority objects are detached from hostile caller state, and execution records can link claims to evidence under run and task scope.

That value depends on honest boundaries. The host must authenticate people, isolate workers, protect credentials, own durable storage, and prevent or monitor bypass paths. External adapters remain trusted components, evidence links do not prove truth, and project tests do not substitute for an independent audit. With those limitations stated, Maqam provides an inspectable building block for applications that need a reviewable line between agent intent and registered side effects.

References

1 Maqam project. *Maqam 0.3.3 source tag*. https://github.com/AjnasNB/maqam/tree/v0.3.3

2 Maqam project. *Complete feature inventory*. https://github.com/AjnasNB/maqam/blob/v0.3.3/docs/FEATURES.md

3 Maqam project. *Maqam Governance Evaluation Suite methodology*. https://github.com/AjnasNB/maqam/blob/v0.3.3/benchmarks/README.md

4 Maqam project. *MGES claim rules*. https://github.com/AjnasNB/maqam/blob/v0.3.3/benchmarks/CLAIMS.md

5 Maqam project. *0.3.1 measured-source performance artifact*. https://github.com/AjnasNB/maqam/blob/v0.3.3/benchmarks/results/2026-07-19-mges-performance-ubuntu24-node24-main-a96413c4.json

6 Maqam project. *0.3.1 measured-source conformance artifact*. https://github.com/AjnasNB/maqam/blob/v0.3.3/benchmarks/results/2026-07-19-mges-conformance-ubuntu24-node24-main-a96413c4.json

7 Node.js project. *process.hrtime.bigint()*. https://nodejs.org/api/process.html#processhrtimebigint

8 npm documentation. *Trusted publishing for npm packages*. https://docs.npmjs.com/trusted-publishers/

9 npm documentation. *Generating provenance statements*. https://docs.npmjs.com/generating-provenance-statements/

10 npm documentation. *Viewing package provenance*. https://docs.npmjs.com/viewing-package-provenance/

11 NIST. *NIST AI 800-2: Practices for Automated Benchmark Evaluations of Language Models, Initial Public Draft*. https://doi.org/10.6028/NIST.AI.800-2.ipd

12 OWASP GenAI Security Project. *OWASP Top 10 for Agentic Applications for 2026*. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

Appendix A. Public implementation surfaces

The tagged package exports policy, gateway, approval, evidence, workflow, skill, function-agent, object-agent, generic tool-adapter, command-line agent, provider-agent, browser-contract, research-source, crawler, release-gate, local-server, error, and boundary utilities. The package includes TypeScript declarations, a local console, command-line entry points, examples, documentation, benchmark schemas, raw benchmark artifacts, and release verification material.

The existence of an export is not a claim that every deployment enables it or that an optional external dependency is present. Public research and browser surfaces require the host to provide the relevant reader, client, executable, credential, or driver where documented.

Appendix B. Claim discipline

Copy derived from this report should preserve the following qualifiers:

- identify 0.3.3 architecture claims with the `v0.3.3` tag and exact commit;
- identify the numerical MGES result as historical 0.3.1 measured-source evidence;
- name the machine, runtime, interval, observations, and exclusions with the point estimate;
- describe 14 of 14 as project-defined conformance fixtures, not a security score;
- state that operations outside registered Maqam boundaries are not governed;
- avoid global, fastest, safest, best, compliant, certified, production-capacity, or formally verified claims without a separate study that supports them.